ASSIGNMENT 1 – TASK 7: Override and Specialise Behaviour

Bloom's Level: K4 – Analyse | Marks: 4

Override ratePerHour() in each vehicle subclass because each category has a different parking rate. @Override asks the compiler to verify the method signature. The overriding method must have the same signature, a compatible (covariant where applicable) return type, and cannot reduce visibility. Static methods are hidden, not overridden. final methods cannot be overridden; final classes cannot be subclassed.

```
abstract class Vehicle {
    private final String regNo;
    Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    public abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException();
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + " " + regNo;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        return regNo.equals(other.regNo);
    }

    @Override
    public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 10.0; }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 30.0; }
}
interface Chargeable {
    double chargingFee(int units);
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 30.0; }
    @Override public double chargingFee(int units) { return units * 12.0; }
}
```

The equality rule in this example deliberately treats any two Vehicle objects with the same registration as equal, matching the supplied demonstration. In a production system, registration numbers should be normalised and validated so one registration cannot be simultaneously registered as different categories. equals() and hashCode() must always agree: equal objects must have the same hash code.

Switch alternative: a switch on vehicle type may be easier for a tiny fixed list, but every new category requires changing the switch. Overriding lets a new subclass supply its own rate without changing calculateFee(). One fee calculation is O(1).